

Echipa:

Pirvanescu Andra-Georgiana

Calotă Denisa-Cosmina

Borozan Iulia-Adriana

Cîmpan Miriam

Semigrupa: 1.2,2.2

Documentație ALU – Unitate Aritmetico-Logică de 8 biți

1. Prezentarea proiectului și obiective

Acest proiect se concentrează pe proiectarea structurală, implementarea și simularea unei Unități Aritmetico-Logice (ALU) de 8 biți folosind limbajul Verilog HDL.

ALU-ul implementat este capabil să execute patru operații fundamentale: adunarea, scăderea, înmulțirea și împărțirea. Un aspect central al acestui proiect a fost adoptarea unei abordări de proiectare strict structurale pentru calea de date (datapath), utilizând componente hardware interconectate precum Sumatorul Parallel (Ripple Carry Adder), multiplexoare și registre interne, toate coordonate de o Unitate de Control dedicată.

Pentru a obține o performanță optimă și eficiență hardware, au fost implementați algoritmi specifici:

- **Algoritmul Booth Radix-4** pentru înmulțirea cu semn, care reduce semnificativ numărul de cicluri de procesare prin gestionarea mai multor biți în fiecare pas.
- **Algoritmul de Împărțire Non-Restoring**, ales pentru implementarea hardware directă și eficiența în evitarea adunărilor redundante în timpul reconstrucției restului parțial.

Obiectivele principale ale proiectului sunt:

- Proiectarea unui ALU de 8 biți modular și structural folosind Verilog.
- Implementarea eficientă a algoritmilor Booth Radix-4 și Non-Restoring Division.
- Proiectarea unui Automat de Stări (FSM) robust care să funcționeze ca Unitate de Control pentru a orchestra fluxurile de date pe baza codurilor de operație primite.

- Validarea funcționalității unității prin testbench-uri și analiză detaliată a formelor de undă.

2. Diagrame de arhitectură și explicații

Arhitectura ALU este împărțită în două blocuri majore interconectate: Calea de Date (Datapath) și Unitatea de Control.

Calea de Date și Arhitectura la Nivel Înalt

Calea de date structurală este construită în jurul unor componente interne pe 9 biți pentru a gestiona corect extensia de semn și depășirile aritmetice pentru datele de intrare de 8 biți. Componentele principale includ:

- **Registrele (A, Q, M):** registre structurale pe 9 biți care păstrează rezultatele intermediare, câțul/multiplicatorul și, respectiv, operandul de intrare.
- **Sumatorul Parallel pe 9 biți (RCA):** funcționează ca unitate de calcul centrală, efectuând atât adunări, cât și scăderi (gestionate prin semnalul `carry_in` care acționează ca activator pentru complementul față de 2).
- **Multiplexorul (MUX 2:1):** selectează între valoarea nemodificată a registrului M și o versiune deplasată a acesteia (pentru operațiile de tip $\pm 2M$ necesare în Booth Radix-4).
- **Contorul de Pași:** un contor pe 3 biți care își ajustează dinamic limita în funcție de operația curentă (3 cicluri pentru înmulțire, 7 pentru împărțire).

Privire de Ansamblu asupra Unității de Control (FSM)

Unitatea de Control este proiectată ca un Automat de Stări Finite care cuprinde exact 13 stări. Acest FSM interpretează codul de operație (`op_code`) pe 2 biți și orchestrează semnalele de control necesare pentru a ruta datele prin calea de date structurală. Stările gestionează inițializările (`INIT_M`, `INIT_Q`), deplasările și evaluările specifice operațiilor (`BOOTH_EVAL`, `DIV_SHIFT`) și sincronizarea finală (`FINISH`).

Descrierea Semnalelor de Control (Flag-uri)

FSM dictează comportamentul ALU prin activarea următoarelor flag-uri de control, fiecare declanșând un comportament structural specific:

- **C0:** Inițializare generală. Încarcă `INBUS` în M, resetează A la 0 și șterge contoarele.
- **C0':** Inițializare specifică Booth. Execută operațiile C0 și forțează suplimentar bitul auxiliar `Q[-1]` la 0.

- **C1:** Încărcare operand. Transferă al doilea operand de pe INBUS direct în registrul Q.
- **C2:** Validarea sumatorului. Actualizează registrul A cu rezultatul operației aritmetice dintre A și M.
- **C2':** Deplasare specifică împărțirii. Declanșează o deplasare logică la stânga a ansamblului A și Q, păstrând bitul de semn original al lui A.
- **C3:** Selecție MUX pentru Booth. Comandă o deplasare la stânga a lui M înainte de intrarea în sumator (multiplicare cu 2).
- **C3':** Corecția câtlui pentru împărțire. Actualizează cel mai puțin semnificativ bit al lui Q (Q[0]) cu valoarea inversată a bitului de semn din A.
- **C4:** Activare scădere. Activează porțile XOR pentru a complementa logic registrul M și setează carry_in la 1, generând complementul față de 2 pentru A - M.
- **C5:** Deplasare specifică Booth. Comandă o deplasare aritmetică la dreapta cu 2 poziții pentru ansamblul A și Q, propagând corect bitul de semn din A.
- **C6:** Controlul iterațiilor. Incrementează contorul de pași.
- **C7 și C8:** Activarea ieșirilor. Plasează conținutul final al registrelor A și Q pe liniile OUTBUS corespunzătoare.
- **END_OP:** Flag de stare finală care indică terminarea cu succes a operației.

3. Listări de cod HDL

Implementarea separă strict logica de control de fluxul de date.

```
module control_unit (
```

```
    input wire clk,
```

```
    input wire reset,
```

```
    input wire begin_op,
```

```
    input wire [1:0] op_code,
```

```
    input wire [2:0] booth_bits,
```

```
    input wire A_sign,
```

```
    input wire count_done,
```

```
output reg c0, c0_prime, c1,  
  
output reg c2, c2_prime, c3, c3_prime,  
  
output reg c4, c5, c6, c7, c8,  
  
output reg end_op  
  
);
```

```
// Starile
```

```
localparam IDLE      = 4'd0,  
  
    INIT_M      = 4'd1,  
  
    INIT_Q      = 4'd2,  
  
    BOOTH_EVAL   = 4'd3,  
  
    BOOTH_SHIFT  = 4'd4,  
  
    DIV_SHIFT    = 4'd5,  
  
    DIV_ADD_SUB  = 4'd6,  
  
    DIV_UPDATE_Q = 4'd7,  
  
    DIV_CORRECT  = 4'd8,  
  
    ADD_FIRST    = 4'd9, // Salveaza primul numar în A  
  
    INIT_M_2     = 4'd10, // Citeste al doilea numar în M  
  
    ADD_SUB_EXEC = 4'd11, // Executa calculul final  
  
    FINISH       = 4'd12;
```

```
reg [3:0] current_state, next_state;
```

```
always @(posedge clk or posedge reset) begin
```

```

    if (reset) current_state <= IDLE;

    else current_state <= next_state;

end

always @(*) begin

    next_state = current_state;

    case (current_state)

        IDLE: if (begin_op) next_state = INIT_M;

        INIT_M: begin

            // Dada e MUL sau DIV, mergem normal. Dada e ADD/SUB, facem flow-ul
special.

            if (op_code == 2'b10 | op_code == 2'b11) next_state = INIT_Q;

            else next_state = ADD_FIRST;

        end

        INIT_Q: begin

            if (op_code == 2'b10) next_state = BOOTH_EVAL;

            else next_state = DIV_SHIFT;

        end

        // Flow Booth

        BOOTH_EVAL: next_state = BOOTH_SHIFT;

        BOOTH_SHIFT: if (count_done) next_state = FINISH; else next_state =
BOOTH_EVAL;

```

```

// Flow Diviziune

DIV_SHIFT: next_state = DIV_ADD_SUB;

DIV_ADD_SUB: next_state = DIV_UPDATE_Q;

DIV_UPDATE_Q: if (count_done) next_state = DIV_CORRECT; else next_state =
DIV_SHIFT;

DIV_CORRECT: next_state = FINISH;


// Flow Adunare/Scadere (3 pasi clari)

ADD_FIRST: next_state = INIT_M_2;

INIT_M_2: next_state = ADD_SUB_EXEC;

ADD_SUB_EXEC: next_state = FINISH;


FINISH: next_state = IDLE;

default: next_state = IDLE;

endcase

end

always @(*) begin

{c0, c0_prime, c1, c2, c2_prime, c3, c3_prime, c4, c5, c6, c7, c8, end_op} = 13'b0;

case (current_state)

INIT_M: begin

if (op_code == 2'b10) c0_prime = 1'b1; else c0 = 1'b1;

```

end

INIT_Q: c1 = 1'b1;

BOOTH_EVAL: begin

case (booth_bits)

3'b001, 3'b010: begin c2 = 1'b1; c3 = 1'b0; end

3'b011: begin c2 = 1'b1; c3 = 1'b1; end

3'b100: begin c2 = 1'b1; c3 = 1'b1; c4 = 1'b1; end

3'b101, 3'b110: begin c2 = 1'b1; c3 = 1'b0; c4 = 1'b1; end

default: begin c2 = 1'b0; end

endcase

end

BOOTH_SHIFT: begin c5 = 1'b1; c6 = 1'b1; end

DIV_SHIFT: c2_prime = 1'b1;

DIV_ADD_SUB: begin c2 = 1'b1; if (A_sign == 1'b0) c4 = 1'b1; end

DIV_UPDATE_Q: begin c3_prime = 1'b1; c6 = 1'b1; end

DIV_CORRECT: if (A_sign == 1'b1) c2 = 1'b1;

ADD_FIRST: c2 = 1'b1;

INIT_M_2: c0 = 1'b1;

ADD_SUB_EXEC: begin

c2 = 1'b1; // Aduna A cu M

```

        if (op_code == 2'b01) c4 = 1'b1; // Daca e scadere, pune Carry=1

    end

    FINISH: begin c7 = 1'b1; c8 = 1'b1; end_op = 1'b1; end

endcase

end

endmodule

```

Implementarea este împărțită în trei blocuri logice principale:

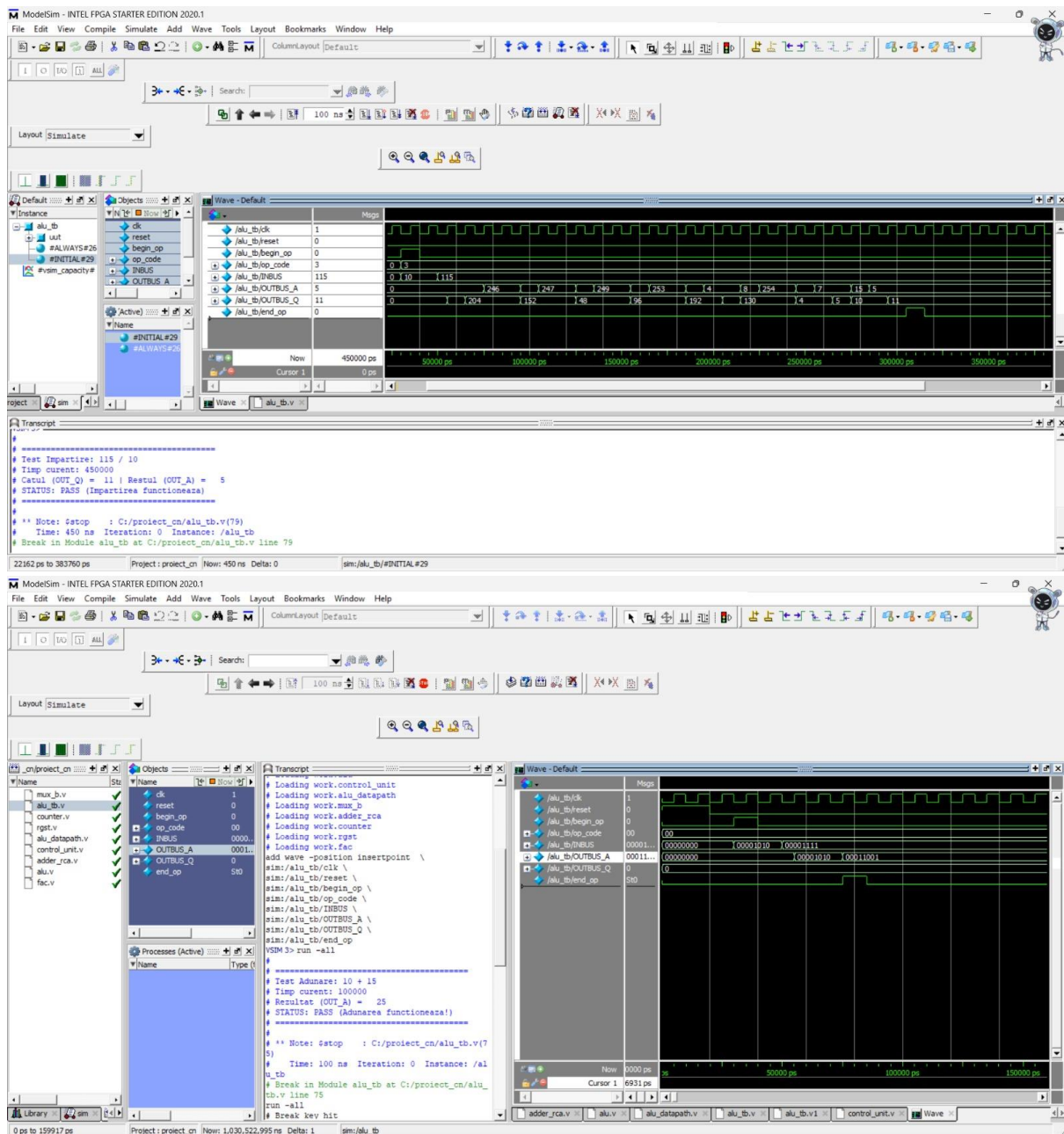
- **1. Unitatea de Control (FSM):** Acționează ca un "creier" care dirijează pașii. Din starea de inițializare, decide dacă execută direct o adunare/scădere sau dacă intră în buclele de iterație pentru algoritmi complecși (Booth sau Împărțire). Buclele se repetă până când contorul semnalează finalizarea (count_done).
- **2. Rutarea Datelor (Registrul A):** Am folosit o abordare pur hardware (multiplexor creat prin operatori ternari ? :). Registrul A nu este modificat haotic; el primește exact ce îi dictează FSM-ul: rezultatul sumatorului (dacă e activ c2), o deplasare cu păstrarea semnului pentru Booth (dacă e activ c5) sau o deplasare logică pentru împărțire (dacă e activ c2_prime).
- **3. Contor Dinamic:** Pentru a economisi resurse, am folosit un singur contor pentru ambii algoritmi. Limita lui se adaptează automat din cod: devine **3** pentru înmulțire (fiindcă Booth procesează mai rapid, 2 biți deodată) și **7** pentru împărțire (care procesează pas cu pas).

4. Descrierea testbench-ului și forme de undă ale simulării

Pentru a asigura fiabilitatea hardware, a fost proiectat un testbench cuprinzător care simulează cicluri de ceas reale și injectează semnalul begin_op împreună cu diverse instrucțiuni op_code.

Cazurile limită, precum aritmetica cu numere negative și înmulțirile cu valori maxime, au fost testate pentru a asigura gestionarea corectă a extensiei de semn în registrele interne





5. Discuția rezultatelor: probleme întâmpinate și rezolvări

Dezvoltarea unui ALU unificat care să includă atât Booth Radix-4, cât și Non-Restoring Division a prezentat câteva provocări unice de sincronizare.

Problema 1: Gestionarea limitelor variabile de iterație

Provocare: Înmulțirea (Booth Radix-4) procesează 2 biți pe ciclu, necesitând mai puține iterații decât împărțirea, care procesează 1 bit pe ciclu. *Rezoluție:* A fost implementată o soluție de rutare dinamică în calea de date. Limita contorului este evaluată structural folosind un operator ternar atribuit unui fir `current_limit` pe baza codului de operație (limita 3 pentru înmulțire, limita 7 pentru împărțire), asigurând ieșirea FSM-ului din stările de buclă la momentul precis necesar.

Problema 2: Diferențe de lățime de bandă și extensia de semn

Provocare: Deși ALU-ul este specificat pe 8 biți, operațiile precum adunarea cu transport sau înmulțirea Booth necesită un bit suplimentar pentru a păstra corect semnul și a preveni erorile de logică în timpul deplasărilor intermediare. *Rezoluție:* Componentele căii de date au fost parametrizate și instanțiate ca module pe 9 biți. La încărcarea INBUS-ului de 8 biți în registrul M de 9 biți, extensia de semn a fost cablată manual (`{INBUS[7], INBUS}`) pentru a menține integritatea aritmetică pe tot parcursul ciclurilor de procesare.

În final, unificarea acestor algoritmi într-un singur FSM și o cale de date partajată a reușit să reducă redundanța hardware. Reutilizarea Sumatorului Parallel pe 9 biți atât pentru ajustările de împărțire, cât și pentru sumările Booth s-a dovedit a fi o alegere arhitecturală eficientă